Lecture Script
Flow Matching

References:

- Lipman et al., *Flow Matching for Generative Modeling* (2022)
- Tong et al., *Improving and Generalizing Flow-Based Generative Models with Minibatch Optimal Transport* (2023)

Outline

1. Summary

Last lecture we looked at diffusion models. The recipe was:

1. Mix data and noise: $z_t = \sqrt{\bar{\alpha}_t}\, x + \sqrt{1 - \bar{\alpha}_t}\, \epsilon$
2. Train a network to predict the noise: $\mathcal{L} = \|\epsilon - \hat{\epsilon}_\theta(z_t, t)\|^2$
3. Generate by denoising many steps backward

Today we'll see that there's an even simpler way to do this:

1. Interpolate between noise and data: $x_t = (1-t)\,\epsilon + t\,x_1$
2. Train a network to predict the velocity: $\mathcal{L} = \|v_\theta(x_t, t) - v_t\|^2$
3. Generate by following the velocity field forward

Same idea—move probability mass from noise to data—but with a straight-line interpolation instead of a noise schedule, and a velocity field instead of a noise predictor.

▹ Let's start from what we know.

2. From denoising to velocity

Recap: the diffusion objects

The two central objects from last lecture:

Diffusion kernel (jump to any noise level):

$$q(z_t \mid x) = \mathcal{N}\Big(z_t;\ \sqrt{\bar{\alpha}_t}\,x,\ (1-\bar{\alpha}_t)\,I\Big) \quad \Longleftrightarrow \quad z_t = \sqrt{\bar{\alpha}_t}\,x + \sqrt{1-\bar{\alpha}_t}\,\epsilon$$

Reverse conditional (denoise one step, given clean data):

$$p(z_{t-1} \mid z_t, z_0) = \mathcal{N}\big(z_{t-1};\ \tilde{\mu}_t(z_t, z_0),\ \tilde{\beta}_t\,I\big)$$

The kernel lets us noise any image to any t in one step. The reverse conditional is the "answer key" for denoising—but requires knowing z_0. The network learns to approximate it without z_0.

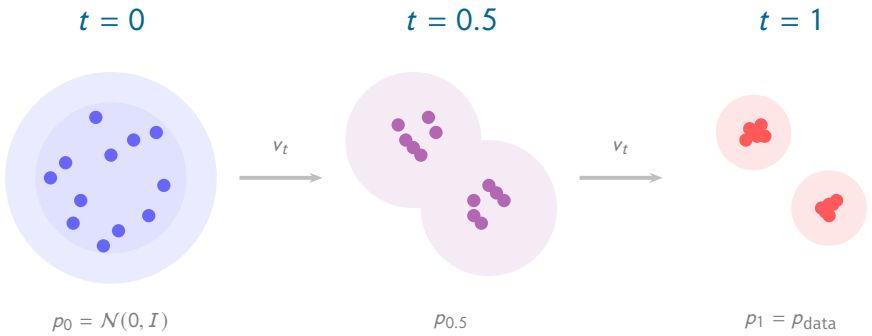
We'll see both of these have direct analogs in flow matching.

The velocity field

Instead of denoising step by step, think of generation as moving particles. Each particle starts at $x_0 \sim \mathcal{N}(0, I)$ and follows a **velocity field** $v_t(x)$:

$$\frac{dx}{dt} = v_t(x)$$

Start a particle at $x_0 \sim \mathcal{N}(0, I)$ and follow the velocity field from $t = 0$ to $t = 1$. If v_t is the right velocity field, the particle ends up as a sample from p_{data}.



This is **flow matching**: learn a velocity field and integrate it forward.

Sampling. In practice, we discretize. Pick a step size Δt (e.g., $\Delta t = 1/N$ for N steps) and integrate forward:

1. Sample $x_0 \sim \mathcal{N}(0, I)$
2. For $t = 0, \Delta t, 2\Delta t, \ldots, 1 - \Delta t$:

$$x_{t+\Delta t} = x_t + \Delta t \cdot v_\theta(x_t, t)$$

3. Return $x_1 \approx$ sample from p_{data}

This is Euler integration. Compare with diffusion sampling: same loop structure, but forward in time ($0 \to 1$) instead of backward ($T \to 0$), and no noise injection—fully deterministic.

▸ OK, so we want to learn a velocity field. How?

3. Conditional flow matching

The marginal velocity is intractable

Ideally, we'd train the network by regression against the true velocity field $u_t(x)$ that transports p_0 to p_1:

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{t,x\sim p_t}\left[\|v_\theta(x,t) - u_t(x)\|^2\right]$$

But what is $u_t(x_t)$? Suppose we knew, for each data point x_1, a conditional velocity $u_t(x_t \mid x_1)$ that transports noise toward x_1, generating a conditional density $p_t(x_t \mid x_1)$. The marginal density at time t is a mixture:

$$p_t(x_t) = \int p_t(x_t \mid x_1)\, q(x_1)\, dx_1$$

What velocity field generates this mixture? Each component $p_t(\cdot \mid x_1)$ is transported by its own $u_t(\cdot \mid x_1)$, weighted by how much it contributes to the density at x_t. The marginal velocity is their weighted average:

$$u_t(x_t) = \int u_t(x_t \mid x_1)\, \underbrace{\frac{p_t(x_t \mid x_1)\, q(x_1)}{p_t(x_t)}}_{q(x_1\mid x_t)}\, dx_1 = \mathbb{E}_{q(x_1\mid x_t)}[u_t(x_t \mid x_1)]$$

The weights are the posterior $q(x_1 \mid x_t)$—by Bayes' rule, the probability that data point x_1 "produced" this x_t. To compute $u_t(x_t)$ at a single point, we'd need to average over all data points, weighted by this posterior. Same intractability as in diffusion: there we needed $p(z_{t-1} \mid z_t)$, which required marginalizing over $p(z_0)$.

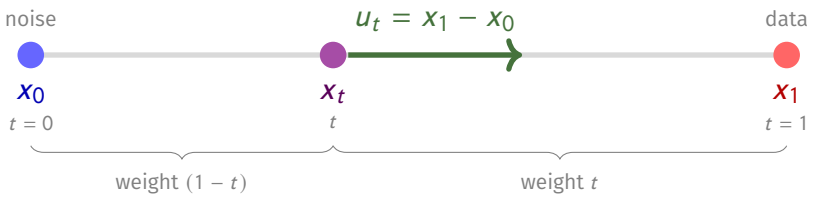
Conditioning makes it tractable

In diffusion, the fix was: condition on the clean data z_0. The reverse step $p(z_{t-1} \mid z_t, z_0)$ was a known Gaussian—the "answer key." We trained a network to approximate it without z_0.

Here the fix is the same: condition on the target data point x_1. Pick a specific data point x_1 and a specific noise sample $x_0 \sim \mathcal{N}(0, I)$. Connect them with a **straight line**:

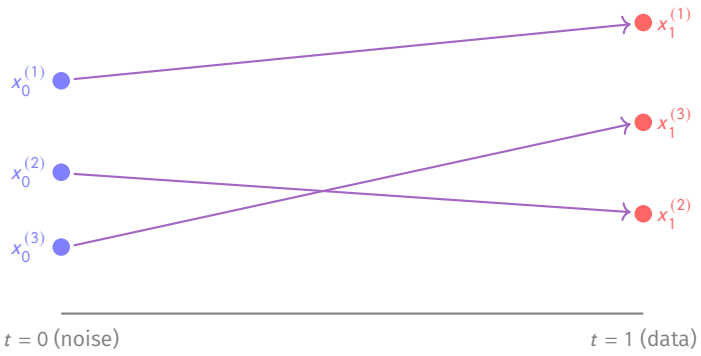
$$x_t = (1 - t)\, x_0 + t\, x_1$$

At $t = 0$: pure noise x_0. At $t = 1$: clean data x_1. The velocity along this straight line is constant: $u_t(x \mid x_1) = dx_t/dt = x_1 - x_0$.



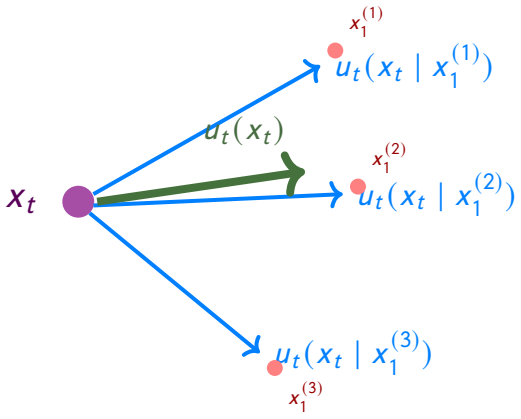
The conditional velocity $u_t(x \mid x_1) = x_1 - x_0$ is constant along the path—it doesn't depend on t. This is our "answer key," the flow matching analog of $p(z_{t-1} \mid z_t, z_0)$ from diffusion.

With many pairs, each (x_0, x_1) traces its own straight line:



Why conditional velocities are enough

At any point x_t, multiple data points contribute conditional velocities. The network learns their weighted average—the marginal velocity:



Each $x_1^{(i)}$ pulls x_t toward itself. Closer data points get higher weight $q(x_1^{(i)} \mid x_t)$. The green arrow is their weighted average.

The **conditional flow matching** (CFM) loss is:

$$\mathcal{L}_{\text{CFM}} = \mathbb{E}_{t,\, x_0,\, x_1}\left[\left\| v_\theta(x_t, t) - \underbrace{(x_1 - x_0)}_{\text{conditional velocity}} \right\|^2\right]$$

This has the **same gradients** w.r.t. θ as the intractable marginal loss $\mathcal{L}_{\text{FM}}$. Why? Because MSE regression converges to the conditional mean of the targets. At a point x_t, the network sees different velocity targets from different pairs—but their conditional mean is exactly the marginal velocity:

$$\mathbb{E}_{q(x_1|x_t)}[\, u_t(x_t \mid x_1)\,] = \mathbb{E}_{q(x_1|x_t)}[\, x_1 - x_0\,] = u_t(x_t)$$

The first equality substitutes our straight-line conditional velocity $u_t(x_t \mid x_1) = x_1 - x_0$. The second is the definition of the marginal velocity from earlier. The network never computes the posterior $q(x_1 \mid x_t)$—regression does the averaging implicitly.

	Diffusion	**Flow Matching**
Marginal (intractable)	$p(z_{t-1} \mid z_t)$	$u_t(x_t)$
Conditional (known)	$p(z_{t-1} \mid z_t, z_0)$	$u_t(x_t \mid x_1) = x_1 - x_0$

Condition on what you're trying to generate (z_0 or x_1), get a trivial target, and regression handles the marginalization.

The training algorithm

For each mini-batch:

1. Sample data $x_1 \sim p_{\text{data}}$
2. Sample noise $x_0 \sim \mathcal{N}(0, I)$
3. Sample time $t \sim \text{Uniform}(0, 1)$
4. Interpolate: $x_t = (1 - t)\, x_0 + t\, x_1$
5. Compute target velocity: $u = x_1 - x_0$
6. Loss: $\|v_\theta(x_t, t) - u\|^2$

That's it. No noise schedule. No $\bar{\alpha}_t$, no β_t. Just a straight line and an MSE loss.

The network v_θ has the same architecture as a diffusion noise predictor (typically a U-Net or transformer)—it takes (x_t, t) and outputs a vector the same size as x_t. Only the training target changes.

▹ Let's put the two methods side by side.

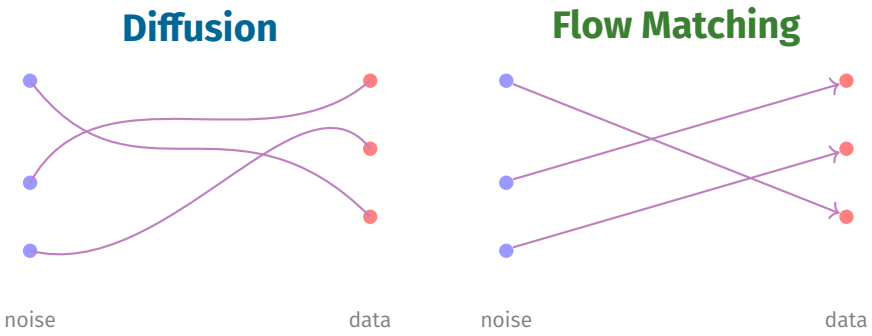
4. Diffusion vs. flow matching

	Diffusion	Flow Matching
Interpolation	$z_t = \sqrt{\bar{\alpha}_t}\, x + \sqrt{1-\bar{\alpha}_t}\, \epsilon$	$x_t = (1-t)\, x_0 + t\, x_1$
Network predicts	noise $\hat{\epsilon}_\theta(z_t, t)$	velocity $v_\theta(x_t, t)$
Training target	ϵ	$x_1 - x_0$
Loss	$\|\epsilon - \hat{\epsilon}_\theta\|^2$	$\|v_\theta - (x_1 - x_0)\|^2$
Generation	$T \to 0$, stochastic	$0 \to 1$, deterministic ODE
Schedule	β_t (needs tuning)	none

Training cost is identical—one forward pass per sample, same architectures. Generation differs: flow matching typically needs far fewer steps.

Why fewer steps?

Diffusion's $\sqrt{\bar{\alpha}_t}$ schedule creates **curved** paths—the signal and noise evolve on different schedules, and undoing this requires many small corrections. Flow matching's linear interpolation gives **straight** paths. The Euler method $x_{t+\Delta t} = x_t + \Delta t \cdot v_\theta$ is exact for constant velocity, so straighter paths need fewer discretization steps.



The connection

The two methods are closely related. The diffusion forward process $z_t = \sqrt{\bar{\alpha}_t}\, x + \sqrt{1-\bar{\alpha}_t}\, \epsilon$ is itself a path from data to noise—just a curved one. Predicting noise $\hat{\epsilon}$ and predicting velocity v are reparameterizations of each other. For any diffusion schedule there's an equivalent flow matching formulation. The linear interpolation $x_t = (1-t)x_0 + t x_1$ is the simplest choice, and gives the straightest paths.

▹ The basics are done. Now some practical tricks, then a key design choice.

5. Practical tricks

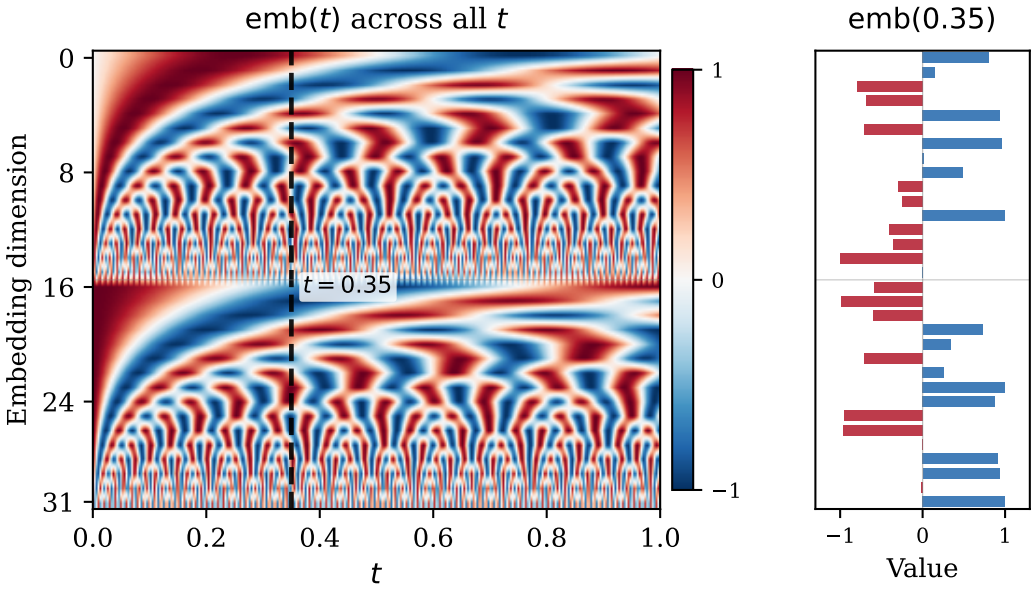
The basic algorithm works, but a few tricks make a large difference in practice. We'll implement each of these in the notebook.

Better architectures. Goes without saying – a better architecture (given your problem/data) will lead to better results!

Time conditioning. The network needs to know t—the same input x_t at different times requires different velocities. A scalar $t \in [0, 1]$ is a poor input: neural networks struggle with single numbers. Instead, project t into a high-dimensional vector using **sinusoidal embeddings** (same idea as positional encodings in transformers):

$$\text{emb}(t) = \Big[\sin(\omega_1 t),\ \cos(\omega_1 t),\ \sin(\omega_2 t),\ \cos(\omega_2 t),\ \ldots\Big]$$

with frequencies ω_k spanning several orders of magnitude. This gives the network a rich, smooth representation of time that it can easily distinguish across different t values.



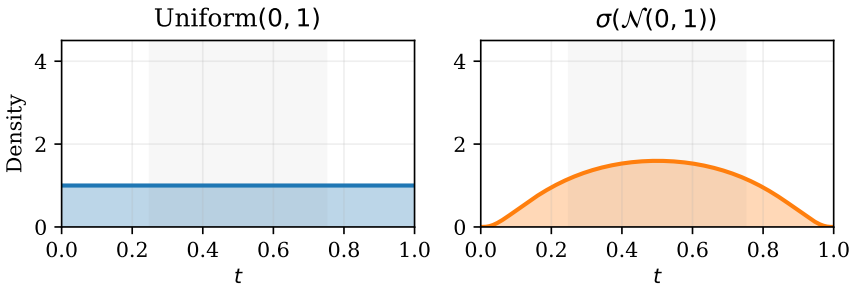
The embedding is then passed through an MLP and used to modulate intermediate features in the network (scale and shift), so the network's behavior smoothly adapts to the noise level.

EMA (exponential moving average). During training, each gradient step updates the weights based on a single mini-batch—noisy by construction. At any given training step, the current weights may be in a temporarily bad region of parameter space. Maintaining a running average smooths this out:

$$\theta_{\text{ema}} \leftarrow \beta\,\theta_{\text{ema}} + (1-\beta)\,\theta, \qquad \beta \approx 0.999$$

Use the EMA weights at inference instead of the raw training weights. The effect is visible: EMA samples are consistently sharper and more coherent.

Logit-normal time sampling. Instead of $t \sim \text{Uniform}(0,1)$, sample $t = \sigma(z)$ where $z \sim \mathcal{N}(0,1)$ and σ is the sigmoid. This concentrates samples near $t = 0.5$. Near $t = 0$ or $t = 1$ the prediction is nearly trivial (output the noise, or output the data). The hard work is in the middle, so spend more training budget there.



σ_{min}. The pure interpolation gives x_1 exactly at $t = 1$—zero noise, which can cause numerical issues. Adding a small floor $\sigma_{\text{min}} \approx 10^{-4}$ keeps a trace of noise at the endpoint:

$$x_t = (1 - (1 - \sigma_{\text{min}})\, t)\, x_0 + t\, x_1$$

▹ One more design choice hidden in the training algorithm.

6. The coupling: how to pair noise and data

Look again at the training algorithm. Each step samples a pair (x_0, x_1) and draws a straight line between them. So far we've sampled $x_0 \sim \mathcal{N}(0, I)$ and $x_1 \sim p_{\text{data}}$ **independently**. But nothing requires that. The CFM loss is valid for any joint distribution $\pi(x_0, x_1)$ whose marginals are p_0 and p_{data}.

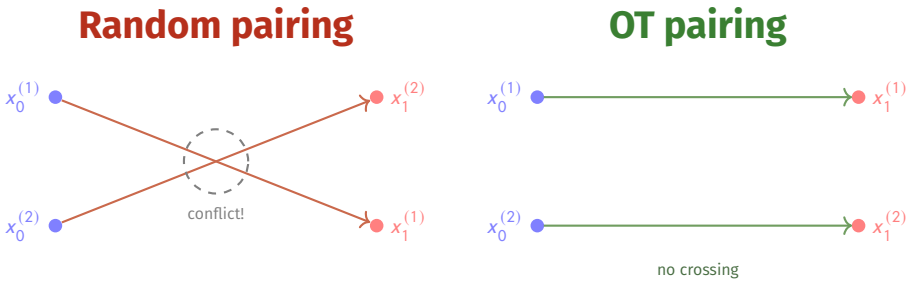
This joint $\pi(x_0, x_1)$ is called a **coupling**—it specifies how noise and data are paired. Two design choices:

1. **How to pair** (the coupling π): independent is simplest, but we can do better.
2. **What to start from** (the base p_0): Gaussian is standard, but p_0 can be anything we can sample from.

Both affect path geometry, and therefore how easy the velocity field is to learn.

Better pairing: mini-batch optimal transport

With independent pairing, conditional paths can **cross**: a noise sample at the top pairs with data at the bottom, while a nearby noise sample goes to the top. At the crossing, the network sees conflicting targets.



The network averages these, producing a sideways velocity—curved marginal paths, higher gradient variance, slower convergence.

Fix: mini-batch OT. Within each mini-batch, pair noise and data to minimize total squared distance:

$$\pi^* = \arg\min_{\pi} \sum_{i=1}^{B} \|x_0^{(i)} - x_1^{(\pi(i))}\|^2$$

where π is a permutation of the mini-batch indices and the cost is squared Euclidean distance $\|x_0^{(i)} - x_1^{(j)}\|^2$. This is a **linear assignment problem**: given a cost matrix of all pairwise distances, find the one-to-one matching that minimizes total cost. Standard solvers (e.g. the Hungarian algorithm) handle this efficiently per mini-batch.

Better starting point: informed base distributions

In diffusion, $p_0 = \mathcal{N}(0, I)$ is baked into the forward process. In flow matching, p_0 is free. If it already captures some structure of the data—spatial correlations, symmetries, known marginals—paths are shorter and the velocity field has less to learn.

This is especially useful in scientific applications where data has known statistical properties. We'll see concrete examples in the notebook.

7. Conditional generation and guidance

To generate x conditioned on some label c (class, text prompt, molecular property, …):

Training. Feed c to the velocity network: $v_\theta(x_t, t, c)$. Randomly drop $c \to \varnothing$ with probability 10–20% so the network also learns the unconditional velocity.

Sampling. Extrapolate between unconditional and conditional predictions (**guidance**):

$$\tilde{v} = v_\theta(x_t, t, \varnothing) + w \cdot \big(v_\theta(x_t, t, c) - v_\theta(x_t, t, \varnothing)\big)$$

$w = 1$: standard conditional generation. $w > 1$: sharper samples, stronger adherence to c, less diversity.

The whole recipe, one more time

Train: Sample $x_1 \sim$ data, $x_0 \sim \mathcal{N}(0, I)$, $t \sim \text{Uniform}(0, 1)$. Set $x_t = (1-t)x_0 + t x_1$. Minimize $\|v_\theta(x_t, t) - (x_1 - x_0)\|^2$.
Generate: Start at $x_0 \sim \mathcal{N}(0, I)$. Step forward: $x_{t+\Delta t} = x_t + \Delta t \cdot v_\theta(x_t, t)$. Return x_1.